

Architecture Design Report

CarbonSense

AI-Based Personal Carbon Footprint Tracking and Sustainability Advisor

Submitted in partial fulfilment for the course of

CSA1012 — Software Engineering

towards the award of the degree of

BACHELOR OF TECHNOLOGY

in Artificial Intelligence and Data Science

Submitted by

Deepak Kumar A

192524422

Under the Supervision of

Dr. K. Anitadavamani

1. System Requirement Analysis

1.1 System Objectives

The problem statement for CarbonSense is to design an AI-based platform that helps individuals measure, understand, and reduce their personal carbon footprint. Based on the Software Requirement Specification (SRS), the primary objectives of the system are:

1. Develop an AI-powered platform to measure and monitor a user's personal carbon footprint from daily activities.
2. Enable users to record activity data such as transportation, electricity usage, food consumption, and waste generation.
3. Accurately calculate carbon emissions using standard emission factors combined with AI-based estimation.
4. Provide personalised, AI-generated sustainability recommendations that help users lower their emissions.
5. Visualise carbon-emission trends and analytics through interactive dashboards.
6. Allow users to set sustainability goals and track progress against them over time.
7. Generate downloadable reports summarising emissions, progress, and environmental impact.
8. Encourage sustained behavioural change through gamification, rewards, and notifications.
9. Give administrators the tools to manage users, emission-factor datasets, and overall system health.

1.2 Functional Requirements

The functional requirements identified from the SRS directly shape the components that the architecture must provide:

- **User Registration & Profile Management:** Register, log in, and manage a personal profile.
- **Activity Data Entry:** Manually enter travel, electricity, food, and waste activity data.

- **Automatic Data Import:** Optionally pull activity data automatically from third-party APIs (e.g. smart meters, travel apps).
- **Carbon Footprint Calculation:** Compute a user's emissions from recorded activities using emission factors and AI-assisted estimation.
- **AI Sustainability Recommendations:** Analyse behaviour patterns and generate personalised suggestions to reduce emissions.
- **Dashboard & Analytics:** Present emission trends, breakdowns, and progress through charts and summaries.
- **Goal Setting & Tracking:** Let users define reduction targets and monitor achievement over time.
- **Notifications:** Send reminders, tips, and milestone alerts via email, SMS, or push notification.
- **Report Export:** Produce and export monthly/yearly emission and progress reports (PDF/CSV).
- **Reward System:** Award points, badges, and achievements for meeting sustainability goals.
- **Admin Management:** Manage users, update emission-factor datasets, and monitor system performance.

1.3 Non-Functional Requirements

The non-functional requirements below directly influence structural and architectural decisions such as service decomposition, communication style, and deployment model:

- **Performance:** Carbon-footprint calculations, dashboard loading, and AI recommendations must respond within 3 seconds.
- **Security:** User data, activity records, and reports must be protected with authentication, authorization, and encryption (in transit and at rest).
- **Reliability:** The system must maintain at least 99.5% uptime with consistent, accurate emission calculations.
- **Availability:** The platform must remain accessible 24/7 through cloud deployment.
- **Scalability:** The system must support a steadily growing user base and increasing volumes of activity data without degrading performance.

- **Maintainability:** A modular design is required so that new AI models, features, or integrations can be added with minimal disruption.
- **Usability:** The interface must be responsive and intuitive across web and mobile platforms.
- **Interoperability:** The system must integrate cleanly with external Carbon Emission APIs, Weather APIs, Maps, and Notification services.

1.4 Key Quality Attributes

The following quality attributes were identified as the primary drivers of the architectural style selection:

Quality Attribute	Relevance to CarbonSense
Scalability	Must support a growing user base and increasing volumes of activity/emission data, especially as AI recommendation and analytics workloads grow independently of core CRUD traffic.
Performance	Carbon calculation and AI recommendation generation must respond within 3 seconds even under concurrent load.
Reliability	Emission calculations must be accurate and consistent; failures in one module (e.g. notifications) must not corrupt core data.
Availability	24/7 cloud access is required; the architecture must tolerate individual component failure without full-system downtime.
Security	Personal activity and location-linked data must be encrypted, authenticated, and access-controlled to meet GDPR/privacy constraints.
Maintainability	The system must evolve to add new AI models, wearable integrations, and emission datasets with minimal impact on existing modules.

2. Selection of Software Architecture

Considering the breadth of functional requirements (user management, activity tracking, AI-driven calculation and recommendations, dashboards, notifications, gamification, reporting, and admin control) together with the non-functional demands for scalability, availability, and independent evolution of the AI engine, a single pure architectural style is insufficient on its own. CarbonSense therefore adopts a hybrid style that blends the clarity of Layered Architecture with the independent scalability of Microservices and the responsiveness of an Event-Driven backbone.

The selected architectural style is a **Hybrid Architecture** that combines:

- **Layered principles at the boundary** — a thin Presentation Layer (web, mobile, and admin clients).
- **Microservices for the core domain** — the Business Logic Layer is decomposed into independently deployable Microservices (Auth, Activity, Carbon Calculation, AI Recommendation, Goal Tracking, Reward, Notification, Reporting, Admin, Data Import).
- **Event-Driven communication for cross-service workflows** — services communicate asynchronously through a central Event Bus / Message Broker for workflows that do not require an immediate response (e.g. triggering AI recommendations after an activity is logged, sending notifications after a goal milestone).
- **API Gateway (Service-Oriented principle)** — a single API Gateway handles authentication, routing, and rate-limiting for all synchronous client requests.

2.1 Justification for the Hybrid (Layered + Microservices + Event-Driven) Architecture

- **Separation of Responsibilities:** The Presentation, Business Logic, and Data layers remain conceptually separated (as in classic Layered Architecture), giving the system a clear, easy-to-understand structure that satisfies the SRS requirement for a modular design.
- **Independent Scalability:** Because AI recommendation workloads, carbon-calculation workloads, and notification workloads have very different resource and scaling profiles, decomposing the business logic into microservices allows each to be

scaled independently — e.g. scaling the AI Recommendation Engine during peak usage without over-provisioning the Auth Service.

- **Support for Asynchronous, AI-heavy Workflows:** Activity logging, footprint calculation, recommendation generation, goal updates, and notifications form a natural event chain. An event-driven backbone lets these steps happen asynchronously, keeping the user-facing request fast (well within the 3-second requirement) while heavier processing happens in the background.
- **Maintainability & Extensibility:** Each microservice can be developed, tested, deployed, and maintained independently. New services (e.g. a future "Carbon Offset Marketplace" or "Wearable Integration" service) can be added without touching existing services — directly supporting the SRS's maintainability and future-enhancement requirements.
- **Centralised Security Enforcement:** The API Gateway centralises authentication/authorization, TLS termination, and rate-limiting, simplifying how security policy is enforced and reducing the attack surface exposed to individual services.
- **Fault Isolation & Availability:** If one non-critical service (e.g. Notification Service) fails or is degraded, the Event Broker retains the event for later delivery, and the rest of the system (login, activity logging, calculation) continues to function — supporting the 99.5% uptime and 24/7 availability targets.

This hybrid approach was chosen over a purely monolithic Layered Architecture because CarbonSense's AI-driven, integration-heavy nature (multiple external APIs, background recommendation processing, notification fan-out) benefits significantly from independent service scaling and asynchronous processing — advantages a single-tier layered monolith cannot easily provide.

3. Software Architecture Diagram

The diagram below illustrates the complete CarbonSense system architecture, including the presentation clients, API Gateway, the microservices that implement the business logic, the event bus that connects them asynchronously, the data storage layer, and the external third-party systems the platform integrates with.

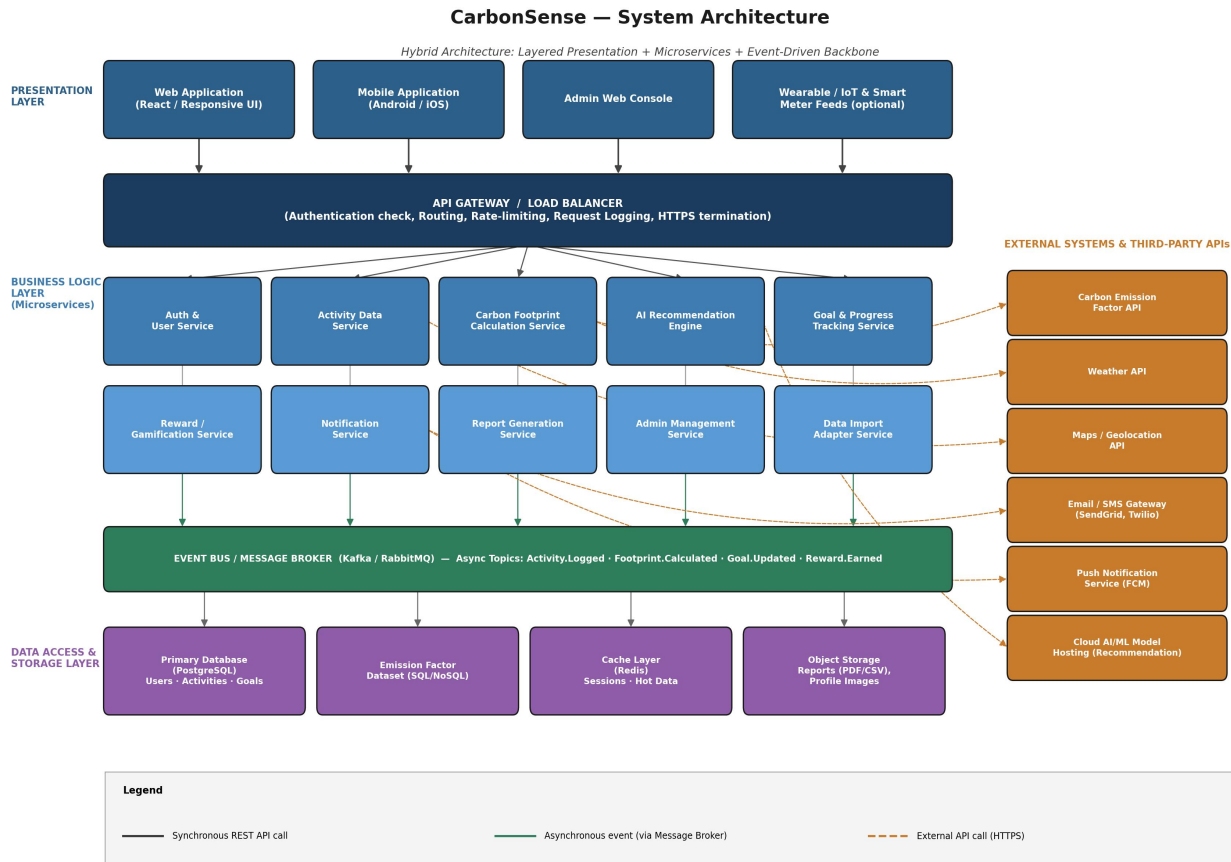


Figure 1: CarbonSense — High-Level System Architecture (Hybrid: Layered + Microservices + Event-Driven)

4. Components and Their Interactions

4.1 Component Descriptions and Responsibilities

Presentation Layer

- **Web Application:** The primary client used by end users to register, log activities, view dashboards, set goals, and download reports. Built as a responsive single-page application.

- **Mobile Application:** Native/cross-platform app offering the same functionality as the web client, with support for on-the-go activity logging and push notifications.
- **Admin Web Console:** A restricted interface used by administrators to manage users, update emission-factor datasets, and monitor system health.
- **Wearable / IoT Feeds:** Optional integration point for wearables and smart-meter data feeds that can automatically supply activity data.

API Gateway / Load Balancer

The single entry point for all client requests. It validates authentication tokens, routes requests to the appropriate microservice, enforces rate limits, performs HTTPS termination, and logs requests for auditing. It shields internal services from direct external exposure.

Business Logic Layer (Microservices)

- **Auth & User Service:** Handles registration, login, JWT issuance/validation, and role-based access control for users and administrators.
- **Activity Data Service:** Receives and stores activity records (travel, electricity, food, waste) entered manually or imported automatically.
- **Carbon Footprint Calculation Service:** Computes carbon emissions from activity records using emission factors and AI-assisted estimation models; publishes a Footprint.Calculated event once complete.
- **AI Recommendation Engine:** Analyses a user's historical activity and emission data using machine-learning models to generate personalised sustainability suggestions.
- **Goal & Progress Tracking Service:** Lets users create emission-reduction goals and tracks progress against recorded footprint data.
- **Reward / Gamification Service:** Awards points, badges, and achievements based on goal completion and sustained sustainable behaviour.
- **Notification Service:** Sends reminders, tips, and milestone alerts through email, SMS, or push notification channels, subscribing to relevant events on the Event Bus.
- **Report Generation Service:** Aggregates emission and progress data into downloadable monthly/yearly reports (PDF/CSV).

- **Admin Management Service:** Provides administrators with tools to manage users, update the emission-factor dataset, and view system-level monitoring data.
- **Data Import Adapter Service:** Connects to optional third-party sources (smart meters, travel/transit apps) to automatically import activity data into the Activity Data Service.

Event Bus / Message Broker (Kafka / RabbitMQ)

Acts as the asynchronous backbone of the system. Services publish domain events such as Activity.Logged, Footprint.Calculated, Goal.Updated, and Reward.Earned to specific topics; interested services subscribe and react independently. This decouples services, enables reliable and replayable delivery, and keeps user-facing requests fast by moving heavier downstream processing (recommendation generation, notification dispatch, reward evaluation) off the synchronous request path.

Data Access & Storage Layer

- **Primary Database:** A relational database (PostgreSQL) storing structured, transactional data: users, activity records, goals, and reward history, with ACID guarantees for critical operations.
- **Emission Factor Dataset:** A dedicated dataset of standard emission factors used by the Carbon Footprint Calculation Service, maintained and updated by administrators.
- **Cache Layer:** An in-memory cache (Redis) used for session storage, frequently accessed dashboard data, and API response caching to reduce database load and latency.
- **Object Storage:** Stores unstructured files such as generated PDF/CSV reports and user profile images, accessed via pre-signed URLs.

External Systems and Third-Party APIs

- **Carbon Emission Factor API:** Supplies standardised emission coefficients used in footprint calculations.
- **Weather API:** Supplies contextual data (e.g. for heating/cooling estimation) that can refine emission calculations.
- **Maps / Geolocation API:** Supports distance and route calculations for transportation-based emission estimates.

- **Email / SMS Gateway:** Third-party providers (e.g. SendGrid, Twilio) used by the Notification Service to deliver email and SMS alerts.
- **Push Notification Service:** Delivers push notifications to mobile clients (e.g. Firebase Cloud Messaging).
- **Cloud AI/ML Model Hosting:** Optional managed infrastructure for training and serving the AI recommendation models at scale.

4.2 Communication and Data Exchange

Components interact through three complementary mechanisms:

1. Synchronous REST APIs — used for immediate, request/response interactions: client login, activity CRUD operations, dashboard queries, and goal management. The API Gateway routes these calls to the responsible microservice.
2. Asynchronous events (Event Bus) — used whenever an action should trigger follow-on processing without blocking the user, e.g. once an activity is logged and its footprint calculated, the AI Recommendation, Reward, and Notification services react to the resulting events at their own pace.
3. External API calls (HTTPS) — services reach the Carbon Emission Factor API, Weather API, Maps API, notification gateways, and AI/ML hosting platforms over HTTPS, with responses cached where appropriate to reduce latency and external call volume.

4.3 Overall System Workflow

A typical end-to-end interaction illustrates how the components cooperate:

1. A user logs in through the Web or Mobile Application; the request passes through the API Gateway, which validates the JWT with the Auth & User Service.
2. The user records a daily activity (e.g. a car trip). The Activity Data Service stores the record and publishes an Activity.Logged event to the Event Bus.
3. The Carbon Footprint Calculation Service consumes the event, retrieves relevant emission factors (and, where needed, Weather/Maps API data), computes the emission value, stores it, and publishes a Footprint.Calculated event.

4. The AI Recommendation Engine consumes this event, analyses the user's updated activity history, and generates a personalised sustainability suggestion, which becomes available on the user's dashboard.
5. The Goal & Progress Tracking Service consumes the same event to update progress toward the user's active goals; if a milestone is reached, it publishes a Goal.Updated event.
6. The Reward / Gamification Service reacts to Goal.Updated by awarding points or badges, publishing a Reward.Earned event.
7. The Notification Service subscribes to milestone and reward events and sends the user a confirmation via push notification, email, or SMS.
8. At any time, the user can request the Report Generation Service to compile a formatted report, stored in Object Storage and made available for download.
9. Administrators use the Admin Web Console, via the Admin Management Service, to manage users and keep the Emission Factor Dataset current.

5. Discussion of Quality Attributes

This section evaluates how the hybrid Layered + Microservices + Event-Driven architecture addresses each of the quality attributes identified in Section 1.4.

5.1 Scalability

- **Independent service scaling:** Each microservice (Activity Data, Carbon Calculation, AI Recommendation, Notification, etc.) can be scaled horizontally and independently. The AI Recommendation Engine — the most compute-intensive component — can be scaled up during peak usage without provisioning extra capacity for lighter services such as Auth.
- **Asynchronous load absorption:** The Event Bus decouples producers from consumers, allowing services to process events at their own pace. This absorbs bursts in activity logging (e.g. many users logging commutes at the same time) without overwhelming downstream services.
- **Load balancing and caching:** The API Gateway can load-balance requests across multiple instances of each service, and the cache layer reduces repeated load on the primary database.

5.2 Security

- **Centralised authentication:** All client requests pass through the API Gateway, which validates authentication tokens and enforces authorization before any request reaches a microservice — reducing the attack surface.
- **Data encryption:** User credentials, activity data, and reports are encrypted both in transit (HTTPS/TLS) and at rest in the database and object storage, satisfying GDPR-aligned privacy constraints noted in the SRS.
- **Service-level access control:** Each microservice only has access to the data it needs (principle of least privilege), limiting the blast radius of a compromised service.

5.3 Performance

- **Fast synchronous paths:** Synchronous calls (login, activity CRUD, dashboard reads) are handled directly by lightweight services and cached where possible, keeping response times within the 3-second target.
- **Offloaded heavy processing:** Heavier work — AI recommendation generation, report compilation, notification dispatch — is moved off the request path via the Event Bus, so it does not block the user-facing response.
- **Caching:** The Redis cache layer serves frequently accessed dashboard and session data without round-tripping to the primary database.

5.4 Reliability

- **Fault isolation:** A failure in one microservice (e.g. Notification Service) does not cascade to others; core functions such as login and activity logging continue to operate.
- **Guaranteed event delivery:** The Event Bus persists events, so if a consuming service (e.g. AI Recommendation Engine) is temporarily unavailable, events are retried and processed once it recovers — no data is lost.
- **Consistent core data:** The relational Primary Database provides ACID guarantees for critical operations such as recording activities and updating goals, ensuring emission data stays consistent and accurate.

5.5 Maintainability

- **Modular codebases:** Each service has a small, focused codebase that is easier to understand, test, and modify. Teams can update the AI Recommendation Engine, for instance, without touching the Reward Service.
- **Clear boundaries:** Well-defined APIs and event contracts between services minimise inter-service coupling, so internal changes rarely ripple outward.
- **Extensibility:** New services (e.g. a future Wearable Integration Service or Carbon Offset Marketplace) can be introduced by subscribing to existing events without modifying current services.

5.6 Availability

- **Redundant service instances:** Multiple instances of each microservice can run behind the API Gateway/load balancer; if one instance fails, traffic is redirected to healthy instances.
- **Cloud-native resilience:** Cloud deployment across availability zones, combined with a highly available Event Broker configuration, supports the SRS's 24/7 availability and 99.5% uptime requirements.

6. Justification of Architectural Decisions

6.1 Rationale Behind the Architectural Choices

The decision to combine Layered principles, Microservices, and an Event-Driven backbone for CarbonSense is driven by the need for a system that is:

- **Scalable to a growing user base:** CarbonSense is expected to grow in both user count and the volume of activity/emission data over time. Microservices allow individual components — particularly the AI Recommendation Engine and Notification Service — to scale independently of the rest of the system.
- **Reliable for a data-sensitive domain:** Because emission calculations and AI recommendations directly influence user trust and behaviour, the architecture isolates failures so that a problem in one service (e.g. Report Generation) cannot bring down core functionality such as activity logging or footprint calculation.
- **Adaptable to an evolving domain:** Sustainability tracking is a domain that will keep evolving — new emission categories, new AI models, new external data sources

(wearables, smart meters, carbon-offset marketplaces). The modularity of microservices and the extensibility of the event-driven backbone make it straightforward to add these without large-scale rewrites.

- **Well suited to AI-driven, asynchronous workflows:** AI-driven recommendation generation is naturally asynchronous — it does not need to complete before the user sees confirmation that an activity was logged. The event-driven design lets this and other background processing happen without impacting perceived responsiveness.

6.2 Trade-offs Considered

While the hybrid approach offers strong scalability and maintainability, it introduces trade-offs that were consciously accepted:

- **Increased operational complexity:** Running and coordinating multiple independent services is inherently more complex than a single monolithic layered application. It requires infrastructure for service discovery, centralized logging, and distributed monitoring.
- **Higher operational overhead:** Deploying, containerizing, and monitoring several services requires more DevOps effort than a single deployable unit, including CI/CD pipelines for each service.
- **Eventual consistency:** Because services own their own data and communicate via events, some data (e.g. a user's updated goal progress) may be briefly out of sync across services until the relevant event is processed — an eventual-consistency trade-off accepted in exchange for decoupling and responsiveness.
- **Inter-service communication overhead:** Asynchronous messaging avoids tight coupling but adds serialization, network, and message-broker latency compared to simple in-process function calls in a monolith.

For a system with CarbonSense's scale and long-term evolution requirements, these trade-offs are outweighed by the gains in scalability, fault isolation, and maintainability. For a much smaller, short-lived project, a simpler monolithic Layered Architecture might have been the more pragmatic choice — a point considered but set aside given the assignment's expectation of a production-oriented design.

6.3 Support for Future Enhancements, Integration, and Long-Term Maintainability

- **Future Enhancements:** New capabilities identified in the SRS's validation section — multilingual support, wearable integration, offline mode, and a carbon-offset marketplace — can each be implemented as new services or client features that subscribe to existing events (e.g. `Activity.Logged`) without modifying the services already in production.
- **System Integration:** The API Gateway offers a single, standardised interface for new external partners (utility companies, government emissions registries, corporate sustainability platforms) to integrate with CarbonSense, while the Event Bus makes it easy to plug in new data sources (smart meters, wearables) by having them publish to existing topics.
- **Long-Term Maintainability:** Because each microservice is small, independently deployable, and loosely coupled through well-defined APIs and events, individual teams can update, refactor, or replace a service (for example, upgrading the AI model behind the Recommendation Engine) without affecting the rest of the platform — reducing technical debt and supporting the system's growth well beyond the initial semester-project scope.